

4.4 Bag of N-Grams

Counts of short phrases keep a little local order.

1. Keep a little word order

One-hot and bag of words treat each token as an island. There is no phrase, and (in BoW) no order.

Bag of n-grams (BoN) is the cheap fix: count **contiguous chunks** of (n) tokens instead of, or in addition to, single words.

An n-gram is that chunk. Unigrams are (n=1) (ordinary BoW). Bigrams are (n=2). Trigrams are (n=3).

The vocabulary (V) is now the set of **distinct n-grams** in the training corpus. Each document is a count vector of length (|V|).

Same four documents:

D1 Dog bites man.

D2 Man bites dog.

D3 Dog eats meat.

D4 Man eats food.

Bigrams in this corpus:

{dog bites, bites man, man bites, bites dog, dog eats, eats meat, man eats, eats food}

Eight columns. (D_1) and (D_2) finally split:

Doc	Bigrams	Vector
(D_1)	dog bites, bites man	[1 1 0 0 0 0 0 0]
(D_2)	man bites, bites dog	[0 0 1 1 0 0 0 0]
(D_3)	dog eats, eats meat	[0 0 0 0 1 1 0 0]
(D_4)	man eats, eats food	[0 0 0 0 0 0 1 1]

“Dog bites man” and “man bites dog” are no longer the same point. That is the whole point of the note.

In the literature this is also called **n-gram feature selection**.

2. What you gain

You buy a slice of **context**. *not good* is one feature, not two independent counts. *New York* is one unit. *bites man* is not *man bites*.

Documents that share n-grams sit closer in Euclidean space than documents that only share unigrams by accident. The vector space is still sparse and discrete, but it is less blind to local syntax.

In sklearn you do not write a new class. `CountVectorizer(ngram_range=(1, 2))` stacks unigrams and bigrams. That mix is the usual production setting: keep the word *good* and the phrase *not good*.

3. What it costs

Dimension explodes. A vocabulary of 20,000 unigrams can yield hundreds of thousands of bigrams. Most never repeat. Sparsity gets worse, not better. Larger (n) means more context and a thinner matrix.

OOV is still there. A new word, or a new pair of known words in a new order, has no column. BoN does not invent features at test time.

Long-range order is still gone. A window of 2 or 3 will not tell you that *shone* at the end of a sentence refers to *hull* at the start. That is a job for later models (CNNs, RNNs, transformers).

4. How to choose (n)

Choice	Use when
Unigrams only	Strong word-level signal, tiny data
<code>(1, 2)</code>	Default for sentiment, spam, topic
<code>(1, 3)</code>	Phrases matter and you have enough documents to fill the counts
<code>(n \ge 4)</code>	Rarely. Most 4-grams appear once.

Always cap features (`max_features`) or demand a minimum document frequency. A 4-gram that appears in one training email is a name, not a pattern.

Note **4.5** keeps these discrete features but **reweights** them so common words do not dominate.

A negation example. Unigrams for “not good” are `{not, good}`. A linear model can assign *good* a positive weight and lose the review. The bigram `not good` is its own column. That is why sentiment labs almost always set `ngram_range=(1, 2)` before they try a neural net.

Character n-grams (`analyzer="char"`) are a different tool: they help with typos and morphology (*run / running*). This note is **word** n-grams. Do not mix the two in your head; mix them in a vectorizer only if you have measured a gain.

5. What to report in a lab

When you add bigrams, say three numbers: vocabulary size, non-zero features per document, and validation F1 versus unigrams-only. If $(|V|)$ grew by 10(\times) and F1 did not move, you paid sparsity for nothing. Drop `min_df` up until the F1 holds. That is n-gram feature selection as an experiment, not as a slogan.

6. Practice

1. List the bigrams in "man eats meat" using the toy vocabulary. Which of them are already in the eight-gram set above, and which are new (OOV)?
2. Why is `ngram_range=(1, 2)` usually safer than bigrams alone?
3. You raise (n) from 2 to 5 on a corpus of 1,000 tweets. What happens to sparsity, and why is that a problem for a linear model?